

Convolutional Neural Network (CNN) Workflow

COMPUTER VISION END-TO-END PIPELINE SOP

PART I: IMAGE DATA ACQUISITION & PREPROCESSING

1. Image Dataset Structuring & Verification

- Inspect image resolutions, color channels (**RGB** **Grayscale** **RGBA**), and file formats (.jpg, .png, .tiff).
- Detect corrupt headers, zero-byte files, duplicate images, or severe label imbalance.
- Filter out low-quality/blurry images using Laplacian variance thresholding.

2. Resizing & Normalization Pipeline

- Resolution Standardisation:** Resize images to standard dimensions (e.g., 224x224, 384x384, or 512x512) preserving aspect ratio via padding/cropping.
- Pixel Scaling:** Scale raw pixel intensity from [0, 255] to range [0, 1] or [-1, 1].
- Channel Mean/Std Normalization:** Normalize image tensors using ImageNet parameters or dataset-calculated channel statistics:
Mean: [0.485, 0.456, 0.406], Std: [0.229, 0.224, 0.225]

3. Computer Vision Data Augmentation

- Spatial Transformations:** Random Horizontal/Vertical Flips, Rotation ($\pm 30^\circ$), Affine Translation, Shearing, and Random Cropping.
- Color & Lighting Jitter:** Adjust Brightness, Contrast, Saturation, and Hue (HSV adjustments).
- Advanced Augmentations:** Apply **CutMix** **MixUp** **AutoAugment** or Random Erasing to improve model robustness.

4. High-Performance Data Loader Pipeline

- Split dataset into Stratified Train (70%), Validation (15%), and Test (15%) sets.
- Construct optimized pipelines (**PyTorch DataLoader** / **tf.data**) utilizing multithreaded fetching, batching, GPU memory pinning ('pin_memory=True'), and prefetching.

PART II: ARCHITECTURE SELECTION & FEATURE ENGINEERING

5. Network Backbone Selection

- Custom CNN Construction:** Build modular stacks of 'Conv2D' → 'BatchNorm' → 'ReLU' → 'MaxPool2D' blocks for custom lightweight tasks.
- Transfer Learning Backbones:** Leverage proven architectures (**ResNet-50/101** **EfficientNet-B0..B7** **ConvNeXt** **MobileNetV3**) pre-trained on ImageNet.
- Fine-tuning Strategy:** Freeze early feature extraction layers and unfreeze deep domain-specific convolutional layers.

PART III: MODEL TRAINING & OPTIMIZATION

6. Loss Function & Optimization Strategy

- **Task-Specific Loss:** Categorical Cross-Entropy (Multi-class), Binary Cross-Entropy (Binary), Focal Loss (Imbalanced Classes), or Smooth L1 Loss / IoU Loss (Object Detection).
- **Optimizers:** Configure AdamW or SGD with Momentum (0.9) with weight decay (1e-4) for regularization.
- **Learning Rate Scheduling:** Implement Cosine Annealing, OneCycleLR, or ReduceLROnPlateau.

7. Training Dynamics & Regularization

- Apply Mixed-Precision Training (FP16 / BF16) to accelerate GPU throughput and reduce VRAM footprint.
- Incorporate Spatial Dropout, Batch Normalization / Group Normalization, and Early Stopping (patience=10).
- Track real-time train/val loss and accuracy curves using TensorBoard or Weights & Biases .

PART IV: VISUAL DIAGNOSTICS & EVALUATION

8. Vision Metrics & Error Analysis

- **Classification Metrics:** Top-1 & Top-5 Accuracy, Precision, Recall, F1-Score, Confusion Matrix.
- **Segmentation / Detection Metrics:** mAP@0.5:0.95, Intersection over Union (IoU), Dice Coefficient.
- **Error Audit:** Isolate misclassified images, out-of-distribution inputs, and boundary ambiguity cases.

9. Explainability & Feature Map Visualization

- Generate Grad-CAM / Grad-CAM++ heatmaps to verify spatial activation regions.
- Visualize intermediate convolutional filters and feature map activations to ensure non-spurious feature learning.

PART V: MODEL COMPRESSION & EDGE DEPLOYMENT

10. Optimization, Export & Deployment

- **Compression:** Apply Post-Training Quantization (INT8 / FP16) and structured channel pruning.
- **Model Export:** Convert PyTorch/TensorFlow weights to production formats (ONNX TensorRT OpenVINO TFLite).
- **Benchmarking:** Validate inference latency (ms/frame), throughput (FPS), and VRAM usage on target target hardware (Nvidia Jetson / Edge / Cloud GPU).

CNN PIPELINE VERIFICATION & SIGN-OFF

Computer Vision Lead: _____ Date: _____